

Categories



Adaptive Query Execution: Speeding Up Spark SQL at Runtime



by [Wenchen Fan](#), [Herman van Hövell](#) and [MaryAnn Xue](#)

May 29, 2020 in [Engineering Blog](#)

Share this post



[Read Rise of the Data Lakehouse](#) to explore why lakehouses are the data architecture of the future with the father of the data warehouse, Bill Inmon.

This is a joint engineering effort between the Databricks
Apache Spark engineering team

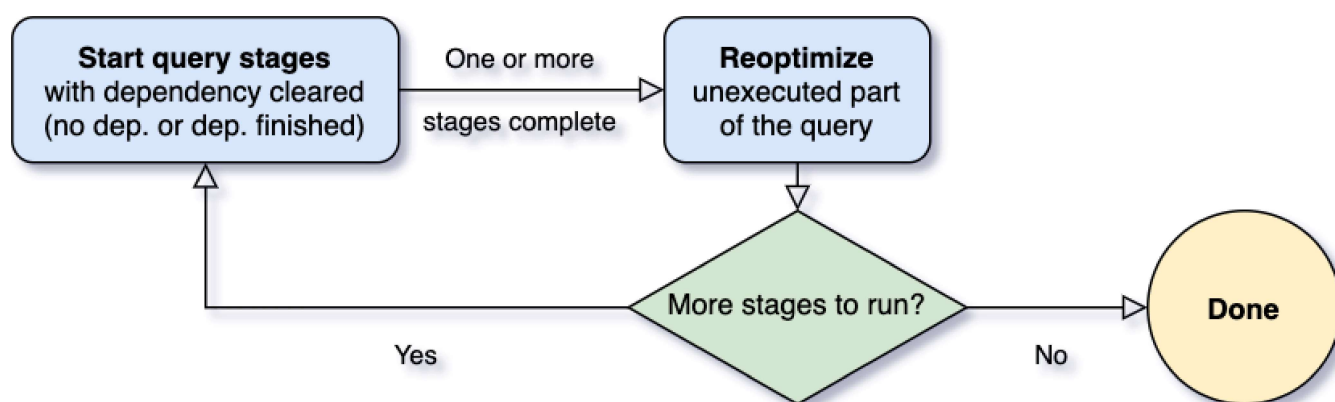
— Wenchen Fan, Herman van Hovell and MaryAnn Xue

[See the AQE notebook](#) to demo the solution covered below or [dive deeper](#) into the inner workings of the Databricks Lakehouse Platform

of the biggest improvements is the cost-based optimization framework that collects and leverages a variety of data statistics (e.g., row count, number of distinct values, NULL values, max/min values, etc.) to help Spark choose better plans. Examples of these cost-based optimization techniques include choosing the right join type (broadcast hash join vs. sort merge join), selecting the correct build side in a hash-join, or adjusting the join order in a multi-way join. However, outdated statistics and imperfect cardinality estimates can lead to suboptimal query plans. Adaptive Query Execution, new in the upcoming Apache Spark™ 3.0 release and available in the Databricks Runtime 7.0, now looks to tackle such issues by reoptimizing and adjusting query plans based on runtime statistics collected in the process of query execution.

The Adaptive Query Execution (AQE) framework

One of the most important questions for Adaptive Query Execution is when to reoptimize. Spark operators are often pipelined and executed in parallel processes. However, a shuffle or broadcast exchange breaks this pipeline. We call them materialization points and use the term "query stages" to denote subsections bounded by these materialization points in a query. Each query stage materializes its intermediate result and the following stage can only proceed if all the parallel processes running the materialization have completed. This provides a natural opportunity for reoptimization, for it is when data statistics on all partitions are available and successive operations have not started yet.



When the query starts, the Adaptive Query Execution framework first kicks off all the leaf stages — the stages that do not depend on any other stages. As soon as one or more of these stages finish materialization, the framework marks them complete in the physical query plan and updates the logical query plan accordingly, with the runtime statistics retrieved from completed stages. Based on these new statistics, the framework then runs the optimizer (with a selected list of logical optimization rules), the physical planner, as well

Now that we've got a newly optimized query plan with some completed stages, the adaptive execution framework will search for and execute new query stages whose child stages have all been materialized, and repeat the above execute-reoptimize-execute process until the entire query is done.

In Spark 3.0, the AQE framework is shipped with three features:

- Dynamically coalescing shuffle partitions
- Dynamically switching join strategies
- Dynamically optimizing skew joins

The following sections will talk about these three features in detail.

Dynamically coalescing shuffle partitions

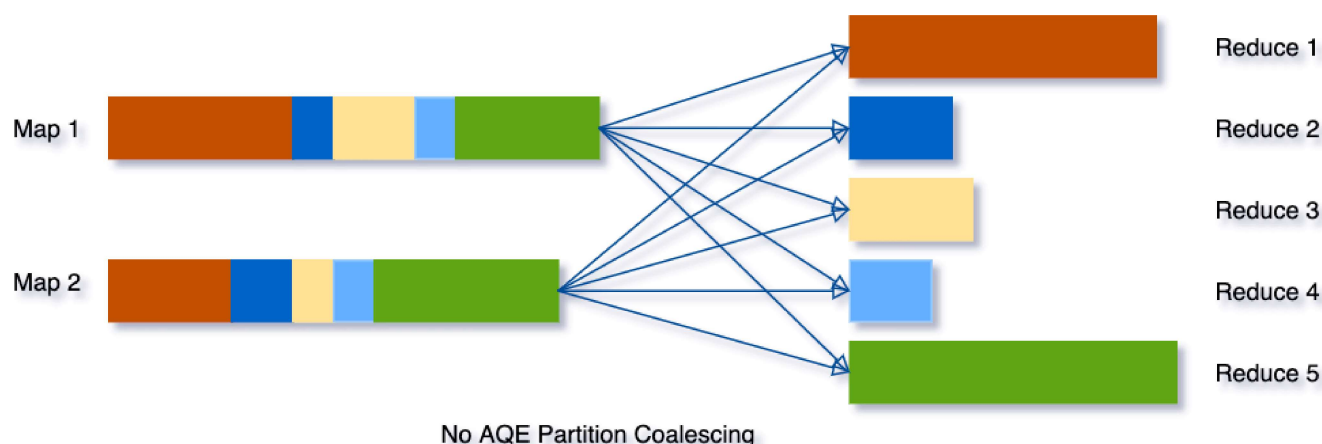
When running queries in Spark to deal with very large data, shuffle usually has a very important impact on query performance among many other things. Shuffle is an expensive operator as it needs to move data across the network, so that data is redistributed in a way required by downstream operators.

One key property of shuffle is the number of partitions. The best number of partitions is data dependent, yet data sizes may differ vastly from stage to stage, query to query, making this number hard to tune:

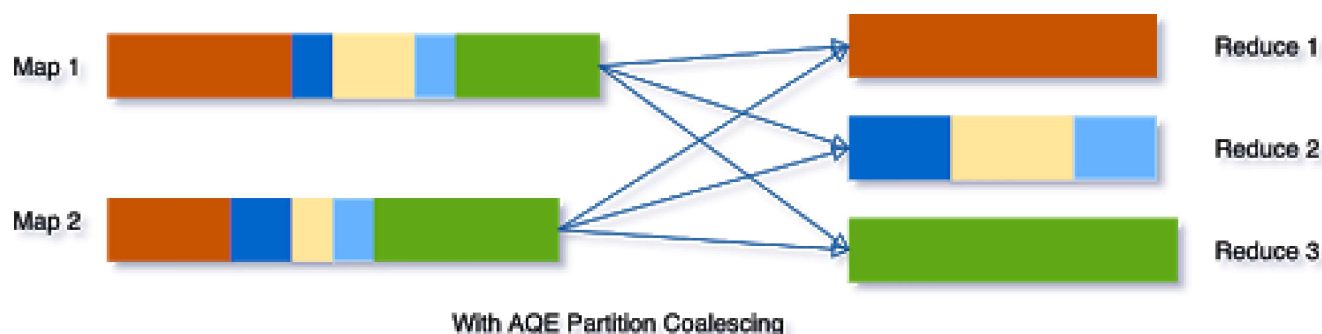
1. If there are too few partitions, then the data size of each partition may be very large, and the tasks to process these large partitions may need to spill data to disk (e.g., when sort or aggregate is involved) and, as a result, slow down the query.
2. If there are too many partitions, then the data size of each partition may be very small, and there will be a lot of small network data fetches to read the shuffle blocks, which can also slow down the query because of the inefficient I/O pattern. Having a large number of tasks also puts more burden on the Spark task scheduler.

To solve this problem, we can set a relatively large number of shuffle partitions at the beginning, then combine adjacent small partitions into bigger partitions at runtime by looking at the shuffle file statistics.

For example, let's say we are running the query `SELECT max(i) FROM tbl GROUP BY j`. The input data `tbl` is rather small so there are only two partitions before grouping. The initial shuffle partition number is set to five, so after local grouping, the partially grouped data is shuffled into five partitions. Without AQE, Spark will start five tasks to do the final



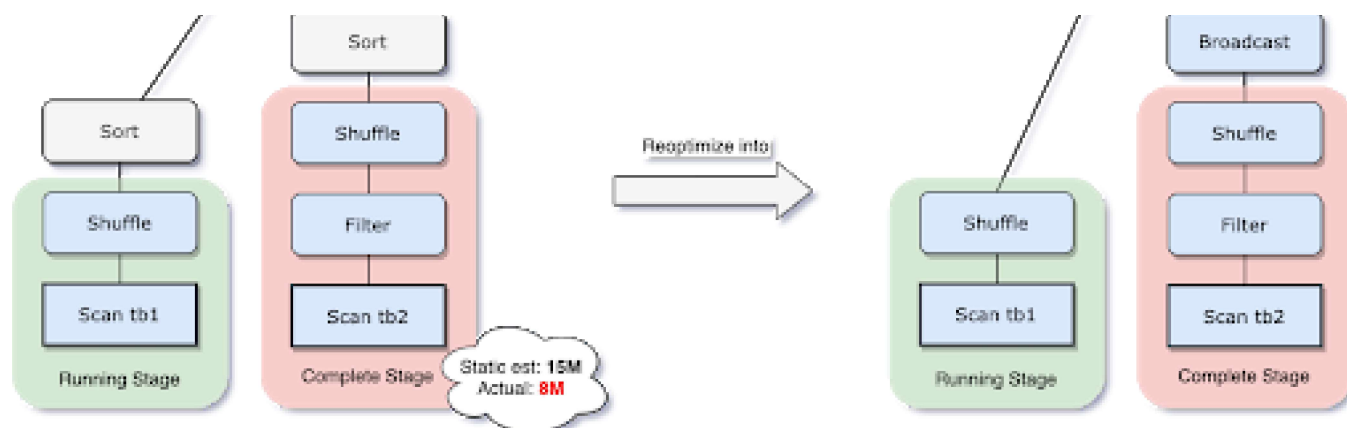
Instead, AQE coalesces these three small partitions into one and, as a result, the final aggregation now only needs to perform three tasks rather than five.



Dynamically switching join strategies

Spark supports a number of join strategies, among which broadcast hash join is usually the most performant if one side of the join can fit well in memory. And for this reason, Spark plans a broadcast hash join if the estimated size of a join relation is lower than the broadcast-size threshold. But a number of things can make this size estimation go wrong — such as the presence of a very selective filter — or the join relation being a series of complex operators other than just a scan.

To solve this problem, AQE now replans the join strategy at runtime based on the most accurate join relation size. As can be seen in the following example, the right side of the join is found to be way smaller than the estimate and also small enough to be broadcast, so after the AQE reoptimization the statically planned sort merge join is now converted to a broadcast hash join.

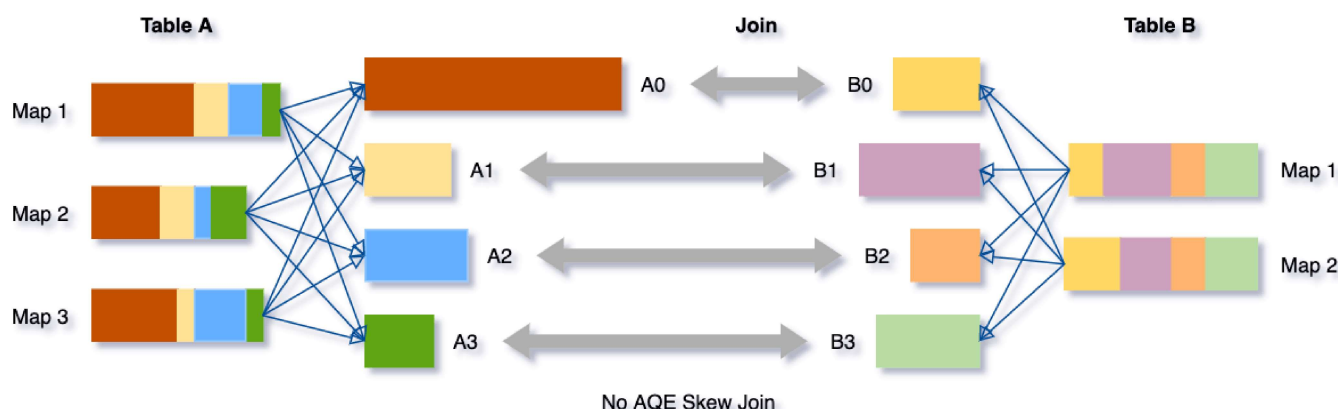


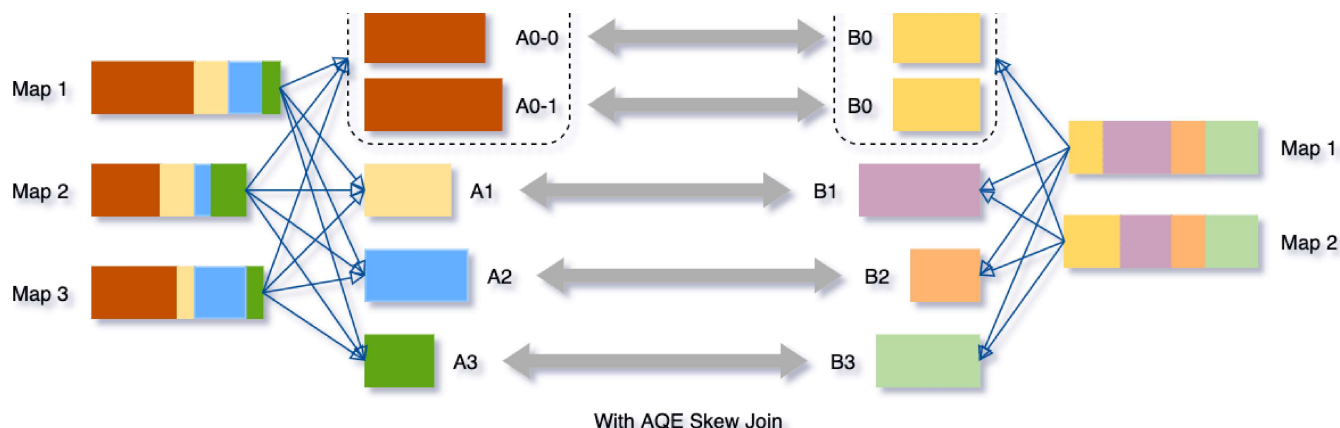
For the broadcast hash join converted at runtime, we may further optimize the regular shuffle to a localized shuffle (i.e., shuffle that reads on a per mapper basis instead of a per reducer basis) to reduce the network traffic.

Dynamically optimizing skew joins

Data skew occurs when data is unevenly distributed among partitions in the cluster. Severe skew can significantly downgrade query performance, especially with joins. AQE skew join optimization detects such skew automatically from shuffle file statistics. It then splits the skewed partitions into smaller subpartitions, which will be joined to the corresponding partition from the other side respectively.

Let's take this example of table A join table B, in which table A has a partition A0 significantly bigger than its other partitions.

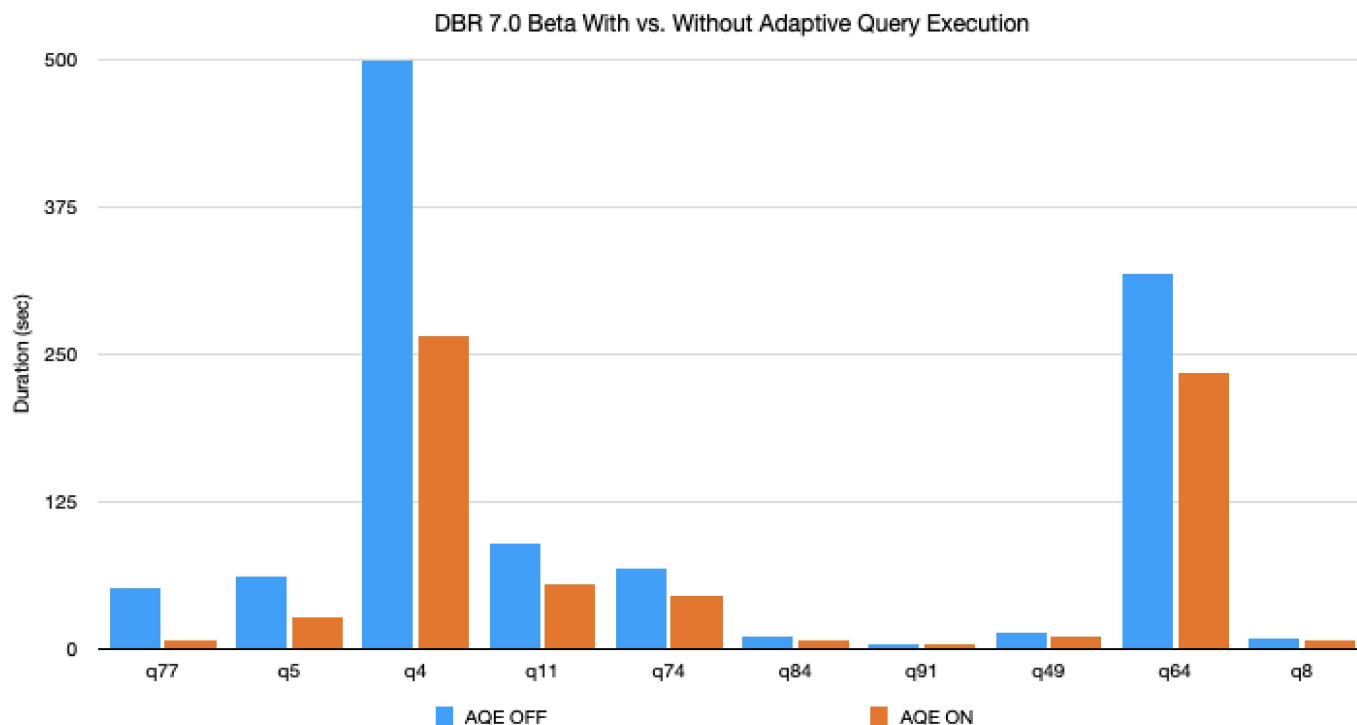




Without this optimization, there would be four tasks running the sort merge join with one task taking a much longer time. After this optimization, there will be five tasks running the join, but each task will take roughly the same amount of time, resulting in an overall better performance.

TPC-DS performance gains from AQE

In our experiments using TPC-DS data and queries, Adaptive Query Execution yielded up to an 8x speedup in query performance and 32 queries had more than 1.1x speedup. Below is a chart of the 10 TPC-DS queries having the most performance improvement by AQE.



seen even greater improvements in production workload in which all three features of AQE are leveraged.

Enabling AQE

AQE can be enabled by setting SQL config `spark.sql.adaptive.enabled` to `true` (default `false` in Spark 3.0), and applies if the query meets the following criteria:

- It is not a streaming query
- It contains at least one exchange (usually when there's a join, aggregate or window operator) or one subquery

By making query optimization less dependent on static statistics, AQE has solved one of the greatest struggles of Spark cost-based optimization — the balance between the stats collection overhead and the estimation accuracy. To achieve the best estimation accuracy and planning outcome, it is usually required to maintain detailed, up-to-date statistics and some of them are expensive to collect, such as column histograms, which can be used to improve selectivity and cardinality estimation or to detect data skew. AQE has largely eliminated the need for such statistics as well as for the manual tuning effort. On top of that, AQE has also made SQL query optimization more resilient to the presence of arbitrary UDFs and unpredictable data set changes, e.g., sudden increase or decrease in data size, frequent and random data skew, etc. There's no need to "know" your data in advance any more. AQE will figure out the data and improve the query plan as the query runs, increasing query performance for faster analytics and system performance.

Learn more about Spark 3.0 in our [preview webinar](#). Try out AQE in Spark 3.0 today for [free on Databricks](#) as part of our Databricks Runtime 7.0.

Try Databricks for free

Get Started



Adaptive Query Execution: Speeding Up Spark SQL at Runtime

May 29, 2020 by [Wenchen Fan](#), [Herman van Hövell](#) and [MaryAnn Xue](#) in [Engineering Blog](#)

Read [Rise of the Data Lakehouse](#) to explore why lakehouses are the data architecture of the future with the father of the data...

[See all Engineering Blog posts](#)



[Why Databricks](#) ✓

[Product](#) ✓

[Solutions](#) ✓

[Resources](#) ✓

[About](#) ✓

Databricks Inc.
160 Spear Street, 13th Floor



© Databricks 2024. All rights reserved. Apache, Apache Spark, Spark and the Spark logo are trademarks of the Apache Software Foundation.

[Privacy Notice](#) | [Terms of Use](#) | [Your Privacy Choices](#) | [Your California Privacy Rights](#)

